

PIXNXT — Backend Decision: Supabase vs MinIO vs Cloudflare vs Golang

Goal: fix 5GB Supabase egress limit, support 200+ photographers + thousands of clients, photo-heavy galleries. Recommendation: Cloudflare R2 (storage, zero egress) + Workers (API/cache) + keep Postgres for DB. MinIO and Golang explained below. Generated Sep 2026.

Criteria	Supabase (current)	MinIO (self-hosted S3)	Cloudflare D1 + Workers + R2 (RECOMMENDED)	Golang backend + Postgres + R2/S3
What it is	All-in-one: Postgres DB + Auth + Storage + Edge Functions	S3-compatible object storage YOU host (VPS/Docker). Storage only, NOT a database.	D1 = SQLite DB, R2 = storage (zero egress), Workers = API server at edge.	Custom API written in Go + Postgres DB + R2/S3 storage. You build everything.
Fixes 5GB egress?	No. Free = 5GB egress, then block/throttle. Photos + DB queries burn it fast.	Yes for files (no Supabase storage egress). But you pay VPS bandwidth instead.	Yes. R2 has ZERO egress fees. Biggest win for photo galleries.	Yes, if files on R2/S3 + CDN. DB egress depends on where you host.
Storage cost (photos/videos)	Free 1GB, then \$0.021/GB. Egress \$0.09/GB — very expensive for galleries.	~\$5-10/mo VPS (Hetzner) for 100s of GB. Cheap but you manage disks/backups.	R2: \$0.015/GB stored, 10GB free. Egress \$0. Best for 200+ users sharing photos.	Same as Cloudflare if you use R2. S3 alone is costly on egress.
Database for 200+ users	Postgres, great. Free limit 500MB. Paid \$25/mo. Your 30+ tables (deliveries, photos, invoices...) fit well.	No DB included. Still need Postgres/Supabase/D1 alongside.	D1: 5GB free, cheap after. Limit: 10GB/DB, SQLite semantics. For PIXNXT scale use Postgres (Neon/Supabase) + D1 only for cache/sessions.	Postgres anywhere (Neon \$0-19, Hetzner \$5). Best control, unlimited scale.
Auth / RLS	Built-in Auth + Row Level Security. You already use it.	No auth. Keep Supabase Auth or Clerk.	Workers has no built-in auth. Keep Supabase Auth (low egress) or Clerk.	You code JWT/auth yourself or use Supabase Auth/Keycloak.
Scalability	Scales to 1000s, but cost jumps. Realtime + SELECT * queries hurt.	Scales with disk. Single VPS = single point of failure unless clustered.	Auto-scales globally (300+ cities). Perfect for client gallery viewers worldwide.	Best raw performance (10K+ req/s). Scales if you add Docker/K8s + load balancer.
Dev effort	Zero — already built.	Low-Medium: Docker + S3 SDK (you already use S3 SDK for R2, same code).	Medium: move hot reads to Workers cache, keep writes on Postgres. You already have workers/ + api/ folders.	High: rewrite Edge Functions + services in Go. Weeks of work.
Maintenance	Zero (managed).	High: updates, backups, disk full, SSL, uptime = your job.	Very low (serverless, no servers).	Medium: manage VPS, DB backups, deploys.
Verdict	Keep for Auth + DB (metadata only). REMOVE files from Supabase Storage (done — you use R2).	Only if you must self-host in India for legal/cost reasons. Not needed — R2 is cheaper.	WINNER for PIXNXT. Files + public gallery via R2 + CDN, API cached on Workers.	Choose only when you hit 1000+ photographers and need max control/speed.

Final plan for PIXNXT (do this): 1) R2 = all photos/videos (already done — keep it). 2) Keep Supabase Free/Pro for Postgres + Auth only (no files, slim SELECTs, pagination). 3) Add Cloudflare CDN in front of R2 + cache public gallery JSON in Workers (1-week cache). 4) Skip MinIO. 5) Skip Golang rewrite until >1000 photographers. This handles 200-2000 users at ~\$5-25/mo instead of Supabase \$25-100+ overages.